

TP Vue.js — Réponses complètes

EMSI — Développement Front-End

Q1 — Fichier .vue (3 sections) vs .tsx (tout mélangé)

Avantages du fichier .vue : - Séparation claire des responsabilités : logique (<script>), structure (<template>), style (<style>) - Plus lisible pour les débutants venant du HTML/CSS classique - Les styles scopés (<style scoped>) n'affectent que le composant courant sans configuration supplémentaire

Inconvénients du .vue : - Nécessite un outillage spécifique (Vite, vue-loader) pour être compris par le navigateur - Le passage de contexte entre les sections peut sembler artificiel pour un développeur JS pur

Avantages du .tsx : - Tout est du JavaScript/TypeScript standard, sans syntaxe spéciale à apprendre - La colocalisation de la logique et du rendu rend les composants très portables - Le typage TypeScript s'applique naturellement partout, y compris dans le JSX

Inconvénient du .tsx : Les styles, la logique et le rendu sont mélangés, ce qui peut nuire à la lisibilité sur de grands composants.

Q2 — Pourquoi React n'a pas de two-way binding natif ?

C'est un **choix philosophique délibéré** de React : le principe du **flux de données unidirectionnel** (*one-way data flow*).

En React, les données descendent toujours du parent vers l'enfant (via les props), et les événements remontent (via les callbacks). Cela rend le comportement **prévisible et traçable** : on sait toujours qui modifie le state et quand.

v-model de Vue est en réalité du **sucre syntaxique** qui cache un :value + @input en coulisses. React préfère rendre cette mécanique **explicite** plutôt que magique, facilitant le débogage dans les grandes applications.

Q3 — Pourquoi tasks.value.push() fonctionne en Vue mais pas en React ?

En Vue 3, ref([]) enveloppe le tableau dans un **Proxy JavaScript**. Un Proxy intercepte toutes les opérations sur l'objet (lecture, écriture, push, splice, etc.). Quand on appelle tasks.value.push(data), le Proxy détecte la mutation et déclenche automatiquement la mise à jour du rendu.

En React, il n'y a pas de Proxy. React compare les **références** pour détecter les changements. Si on fait `tasks.push(data)`, la référence du tableau reste la **même**, React ne détecte aucun changement et ne re-rend pas le composant. D'où l'obligation de créer un **nouveau tableau** avec `[...prev, data]`.

Q4 — `useEffect(fn, [])` vs `onMounted(fn)` : lequel est plus lisible ?

`onMounted` est **plus lisible** car son nom exprime clairement l'intention : "exécute ça quand le composant est monté". Pas besoin de connaître la convention du tableau vide.

`useEffect` avec `[]` est une **convention apprise** : le tableau de dépendances indique à React quand relancer l'effet. Un tableau vide signifie "une seule fois au montage". React a fait ce choix pour unifier la gestion de **tous les effets de bord** (montage, mise à jour, démontage) dans un seul hook, plutôt que d'avoir `componentDidMount`, `componentDidUpdate`, `componentWillUnmount` séparés. C'est plus puissant mais moins intuitif.

Q5 — Fonctions en props (React) vs événements émis (Vue) : qu'est-ce qui est plus proche du HTML natif ?

La **syntaxe Vue** (`@delete`, `@click`, `@toggle`) est plus proche du HTML natif. En HTML, on écoute des événements (`onclick`, `onchange`, `oninput`), pas des fonctions passées comme attributs. Vue reprend exactement ce paradigme : l'enfant **émet** un événement, le parent **l'écoute**.

En React, passer `onDelete={fn}` est techniquement une prop comme les autres — c'est une fonction JavaScript passée en paramètre. C'est cohérent avec la philosophie "tout est JS", mais conceptuellement plus éloigné du modèle DOM natif.

Q6 — Que se passe-t-il si on oublie `@delete` en Vue ?

L'événement est **silencieusement ignoré**. L'enfant appelle `emit('delete', id)`, Vue cherche un listener dans le parent, n'en trouve pas, et ne fait rien — pas d'erreur, pas de crash.

C'est à double tranchant : moins de risque de crash en production, mais les bugs sont plus difficiles à détecter car il n'y a aucun avertissement visible.

Q7 — `useParams()` + `useNavigate()` (React) vs `useRoute()` + `useRouter()` (Vue) : vraiment différent ?

La logique est **fondamentalement identique**. Les deux frameworks séparent la lecture de l'URL actuelle (params, query) et la navigation programmatique. Seuls les noms changent :

Rôle	React Router	Vue Router
Lire les paramètres	<code>useParams()</code>	<code>useRoute().params</code>
Naviguer	<code>useNavigate()</code>	<code>useRouter().push()</code>

La vraie différence : `useRoute()` donne accès à **tout** (params, query, hash, path) en un seul objet, là où React Router les fragmente en plusieurs hooks (`useParams`, `useSearchParams`, `useLocation`).

Q8 — Routes dans le JSX (React) vs fichier de config (Vue) : quel avantage ?

La **séparation de Vue** (`router/index.ts`) offre plusieurs avantages :

- **Vue d'ensemble immédiate** : toutes les routes de l'app sont visibles en un seul endroit
- **Navigation guards globaux** faciles à ajouter (`router.beforeEach(...)`)
- **Meilleure séparation des concerns** : la config du routing ne pollue pas le composant racine
- **Outillage** : Vite peut analyser statiquement le fichier pour optimiser le code splitting

En React, disperser les `<Route>` dans le JSX permet plus de **flexibilité et de dynamisme** (routes conditionnelles selon le state), mais rend l'architecture globale moins lisible.

Q9 — Redux Toolkit vs Pinia : comptage des concepts

	Redux Toolkit	Pinia
Créer le store	<code>createSlice</code> + <code>configureStore</code>	<code>defineStore</code>
Fournir le store	<code><Provider store={store}></code>	<code>app.use(pinia)</code>
Lire le state	<code>useSelector(state => ...)</code>	<code>store.tasks</code>
Déclencher une action	<code>useDispatch()</code> + <code>dispatch(action())</code>	<code>store.addTask()</code>
Total	~5 concepts	~2 concepts

Pinia est significativement plus simple à apprendre et à utiliser au quotidien.

Q10 — `dispatch(addTask(title))` vs `store.addTask(title)` : lequel est plus intuitif ?

`store.addTask(title)` est **plus intuitif** : c'est un simple appel de méthode, comme dans n'importe quel objet JavaScript.

`dispatch(addTask(title))` est plus **cérébral** : il faut comprendre le pattern action → dispatcher → reducer.

Avantage de Redux que Pinia n'a pas pleinement : Les DevTools avec **time-travel debugging**. Comme chaque action Redux est un objet sérialisable et immuable, on peut rejouer, annuler et inspecter chaque mutation du state dans le temps.

Q11 — Tableau comparatif complet React vs Vue 3

Concept	React	Vue 3
State local	<code>const [x, setX] = useState(0)</code>	<code>const x = ref(0)</code>
Two-way binding	<code>value={x} onChange={e => set(e.target.value)}</code>	<code>v-model="x"</code>
Fetch au montage	<code>useEffect(() => { fetch()... }, [])</code>	<code>onMounted(async () => { fetch()... })</code>
Ajouter au state	<code>setItems(prev => [...prev, data])</code>	<code>items.value.push(data)</code>
Boucle template	<code>items.map(i => <li key={i.id}>...)</code>	<code><li v-for="i in items" :key="i.id"></code>
Conditionnel	<code>{condition && <div>...</div>}</code>	<code><div v-if="condition"></code>
Event	<code>onClick={handler}</code>	<code>@click="handler"</code>
Props enfant	<code><Child name={x} /></code>	<code><Child :name="x" /></code>
Callback parent	<code><Child onDelete={handler} /></code>	<code><Child @delete="handler" /></code>
Router lien	<code><Link to='/x'>Lien</Link></code>	<code><RouterLink to='/x'>Lien</RouterLink></code>
Params URL	<code>const { id } = useParams()</code>	<code>useRoute().params.id</code>
Navigate	<code>const nav = useNavigate(); nav('/')</code>	<code>const r = useRouter(); r.push('/')</code>
State global	<code>createSlice + useSelector + dispatch</code>	<code>defineStore + useTaskStore()</code>
Appeler action	<code>dispatch(addTask(title))</code>	<code>store.addTask(title)</code>
Style isolé	CSS Modules / <code>.module.css</code>	<code><style scoped></code>

Concepts IDENTIQUES (nom seul change) : routing (params + navigation), cycle de vie (montage), props parent→enfant, liens de navigation.

Concepts FONDAMENTALEMENT différents : réactivité (Proxy vs immutabilité), two-way binding, communication enfant→parent (emit vs callback prop), templates (directives HTML vs JSX JavaScript).

Q12 — Vue est-il plus “magique” ? Est-ce un avantage ?

Oui, Vue est plus “magique” : `v-model`, les mutations directes via Proxy, les directives `v-for`/`v-if` — Vue fait beaucoup de choses en coulisses.

Avantage : productivité immédiate, code plus concis, courbe d’apprentissage moins raide.

Inconvénient : la magie peut masquer des bugs subtils (effet de bord non tracé, réactivité cassée sur des propriétés imbriquées) et rend le comportement moins prévisible. En React, chaque mise à jour du state est **explicite et intentionnelle**, ce qui facilite le raisonnement dans des applications très complexes et le travail en grande équipe.

Q13 — App e-commerce, 50+ pages, équipe de 10 développeurs : React ou Vue ?

React. Voici pourquoi :

- **Écosystème plus mature** pour des besoins entreprise (Next.js, nombreuses librairies UI)
 - **TypeScript + immutabilité** rendent le code plus robuste et plus facile à auditer en grande équipe
 - **Redux DevTools** avec time-travel debugging précieux pour un dashboard admin complexe
 - **Plus grande communauté** = plus de ressources, plus de développeurs au recrutement
 - **L’explicité de React** réduit les bugs silencieux quand 10 personnes travaillent ensemble
-

Q14 — Débutant sans expérience de framework : React ou Vue en premier ?

Vue. Pour un vrai débutant :

- La **structure .vue** (script + template + style) est intuitive pour quelqu’un venant du HTML/CSS/JS
- Les **directives** (`v-if`, `v-for`, `v-model`) ressemblent à du HTML enrichi
- **Moins de concepts abstraits** à assimiler (pas d’immutabilité, pas de JSX, pas de flux unidirectionnel à comprendre dès le départ)
- **Pinia est bien plus simple** que Redux pour une première approche du state management

Une fois les fondamentaux assimilés avec Vue, passer à React devient bien plus accessible car les concepts sous-jacents sont les mêmes.